MatchPulse — Architecture Report

Date	2026-11-08
Status	Current project report; the semester is still running.
Scope	Software Architectures (CM0639, module 1), shared with Development Methodologies.
Repository	https://github.com/match-pulse/match-pulse

This report describes the current architecture of MatchPulse. It is a dated snapshot, not a final submission. It links to the detailed records rather than duplicating them. The authoritative statements remain the linked records; where this report and a record disagree, the record wins.

1. Executive summary

MatchPulse is a football tournament platform for local competitions. Spectators buy tickets, register for a place, follow live match data and trade prediction points on the match outcome. Points are redeemable for future ticket discounts, and aggregated prediction data is sold to clubs.

The system is a **single Next.js application and a single PostgreSQL database**. Critical business rules are enforced inside PostgreSQL transactions and row-level security, not in the browser or in ad hoc application checks. The architecture is best described as a **modular monolith at the domain and database boundary**, with a layered Next.js delivery shell: one deployable, one database, explicit domain modules in the schema and RPC surface, and executable guards on the application imports.

The selected architecture characteristics are **Reliability, Elasticity, Deployability, Testability and Security**. Reliability and Security are measured for the recorded mechanisms; Elasticity is partially measured; Deployability is measured for local/CI; Testability is measured with one stated injection gap. No production availability, production latency or real-user claim is made.

2. Kata

The current kata is `../kata.md`. It has the four parts the course requires and numbers the requirements K1–K15 so the characteristics can refer to them.

2.1 Description

A platform for managing local football tournaments. Spectators buy tickets, register for a place, follow live match data and trade prediction points on the match outcome. Points are redeemable for ticket discounts. Aggregated, time-stamped prediction data is sold to clubs.

2.2 Users and scale

User	Expected number and pattern	What they do
Spectators	Hundreds concurrently during a match; thousands across a tournament. A planning assumption, not yet measured.	Buy tickets, register, follow the match, trade, redeem rewards.
Tournament staff	Roughly 1–5 per tournament, active at setup, during the match and at result time.	Create matches, set capacity, update scores, finalise results.
Club representatives	A few analysts per club, reading/exporting after matches.	Access and export aggregated prediction data.

Venue capacity bounds participation; the platform must not register more places than the venue allows.

2.3 Requirements

ID	Requirement
K1	Staff create and manage matches and ticket sales.
K2	Staff set a maximum participant capacity per tournament, bounded by the venue.
K3	Spectators buy tickets and receive prediction points.
K4	Spectators register while capacity remains and are told when the tournament is full.
K5	Spectators view the live scoreboard, timer and available market on their phones.
K6	Spectators trade points on HOME/DRAW/AWAY while the match is running, entering and exiting at current odds.
K7	A spectator chooses the points to commit; the market prices the position, and the spectator may exit part or all before the result is finalised.
K8	Exiting settles against current odds: a gain when the implied probability rises, a proportional loss when it falls.
K9	The platform takes no commission on entries, exits or settlement.
K10	Re-entry after closing a position is delayed by an escalating cooldown; exits are never blocked or charged.
K11	Match scores are updated during the game, and staff finalise the result.
K12	Finalisation settles open positions: winning shares pay one point each, losing shares pay nothing.
K13	Spectators redeem points for future ticket discounts.
K14	Club representatives access and export aggregated prediction data, including how the crowd's view changed.
K15	The platform sells aggregated prediction data; points cannot be converted to cash.

2.4 Additional context

- Traffic peaks during matches, with hundreds of concurrent spectators as the current planning assumption.

- Participation must never exceed capacity, including simultaneous requests.
- Balances, positions and trade records must remain consistent under concurrency.
- The data is valuable because it combines the crowd's aggregate position with time.
- Revenue comes from club data sales; points are not cash.
- The first implementable increment covers capacity-safe registration and the live market. Ticket purchase, rewards redemption and club analytics are part of the product vision but are not all implemented.

3. Architecture characteristics

The full derivation, course-vocabulary mapping and SMART table are in `../03-architecture-characteristics.md`. This section summarises the selection and its evidence.

3.1 Selection

Characteristic	Kata points	Why selected	Quantified target	Status
Reliability	K2, K4, K6–K8, K10–K12	Capacity, balances, positions and settlement are concurrency rules; failure corrupts the ledger.	100 concurrent registrations, capacity 32: 32 accepted, 68 refused, 0 transport errors; no negative balance; one-shot resolution; idempotent retries.	Measured for recorded mechanisms; FND-002 open.
Elasticity	K5, K6, traffic context	Match traffic is a step change at kickoff, not a gradual ramp.	p95 < 1,000 ms for 100 concurrent registrations in CI; post-peak resource release unverified.	Partially measured.
Deployability	K1–K14, solo-developer context	A one-person project across two deadlines must deliver without infrastructure work.	Merge to running verified local/CI environment <15 min; zero manual DB steps; migrations only.	Measured local/CI; production unverified.
Testability	K2, K4, K6–K8, K10–K12	The core rules cannot be verified by inspection.	Every enforced invariant has a failable test; suite <3 min; every PR.	Measured; FND-002 open.
Security (implicit)	K1, K13, K14	Staff-only management, the points ledger and club privacy create a trust boundary.	No anonymous private read/mutation; no cross-user read; organizer-only resolution; direct writes denied; no service-role key in the client.	Measured; production hardening and club privacy unverified.

3.2 Course-vocabulary note

The course's consolidated table lists 12 characteristics. It does not list Reliability or Security. The same course discusses Reliability as an operational characteristic and uses Security as the example of an implicit characteristic. This report names them explicitly rather than forcing the domain risk into Fault tolerance or leaving security implicit:

- **Reliability** is the correct name for "the system functions correctly under specified conditions for a specified period." The conditions here are simultaneous requests; the period is the burst.
- **Security** is an implicit cross-cutting characteristic made explicit. The kata does not use the word, but the staff boundary, the points ledger and club privacy require it.

3.3 What is not selected

Performance is not selected separately; the only time target is the burst p95 under Elasticity. Scalability is not selected because the kata states peaks, not growth beyond one deployment. Fault tolerance is not selected because there is no hardware-failure target. Modularity, maintainability, evolvability, simplicity, overall cost and partitioning type are design considerations, not primary success drivers for the first increment. The full rationale is in the characteristics document.

4. Architecture style and module boundaries

4.1 Style

MatchPulse is a **modular monolith** in the sense that matters here:

- one deployable Next.js application;
- one PostgreSQL database;
- explicit domain modules in the schema and RPC surface;
- no network boundary between modules;
- no separate deployment or independent scaling claim.

It is **not** a textbook code-level modular monolith with a `modules/` directory per domain. The Next.js layer is organised by technical role: routes and Server Actions, client components, and shared server utilities. The domain boundaries are enforced primarily in PostgreSQL and by the architecture guard. This distinction is stated explicitly so the label is not overclaimed.

4.2 Domain modules

Module	Tables / functions	Responsibility
Identity and access	`profiles`, auth trigger, RLS policies, grants	Supabase Auth identity, roles, row-level access
Tournaments	`tournaments`, `registrations`, `register_for_tournament`	Capacity-bounded registration and uniqueness
Markets	`markets`, `market_positions`, `market_trades`, `market_entry_cooldowns`, `market_payouts`, `open_trade`, `close_trade`, `resolve_market`, `market_public_trades`	LMSR pricing, entry/exit, cooldown, one-shot resolution
Trade requests	`trade_requests`, request-key logic in the trade RPCs	Idempotent retries and payload mismatch refusal

Module	Tables / functions	Responsibility
Delivery and operations	`/api/health`, `src/lib/logger.ts`, `bun run smoke`, operations runbook	Health, logs, delivery and recovery evidence

4.3 Dependency rules

- Client components call Server Actions; they do not import the server Supabase client.
- Server Actions validate input and call authorized RPCs; they do not mutate tables directly.
- Mutation RPCs are `SECURITY DEFINER`, resolve `auth.uid()`, fix `search_path`, and grant execute only to `authenticated`.
- Direct table writes are revoked; RLS restricts private reads to the owner.
- The service-role key is a local test/administration credential, never an application runtime key.

`scripts/check-architecture.ts` enforces the text-level rules: no service-role key in `src/`, no direct table mutations, no server-client imports from components, no retired client import, and RPC definitions with auth checks and explicit grants. This is a guard, not a proof; the database-backed authorization suite is the behavioral check.

4.4 Why not microservices

The domain has one bounded context, one database and one developer. Splitting it would add network failure modes, distributed consistency and operational work without a requirement that justifies them. The modular monolith gives explicit boundaries and database-enforced invariants at a fraction of that cost. If club analytics becomes a separate workload with its own scaling driver, extraction can be reconsidered; that is a future condition, not a current property.

5. System overview

5.1 Runtime view

1. A browser requests a page from the Next.js App Router.
2. The page resolves the current Supabase Auth session from cookies.
3. Server-rendered data is read through the cookie-based Supabase client; RLS limits private rows to the authenticated user.
4. A mutation is submitted to a Server Action.
5. The action validates the form shape and calls an authorized PostgreSQL RPC.
6. The RPC resolves the caller with `auth.uid()`, locks the relevant rows, enforces the business rule and commits the trade, position, balance and request-key state atomically.
7. The page refreshes and renders the new state.

5.2 Data flow for a trade

`MarketTrader` → `enterPosition` or `exitPosition` → `open_trade` or `close_trade` → market/profile/cooldown/position rows → `market_trades` and `trade_requests`. The client supplies one request key across retries; the database claims the key before the trade, replays a completed request, refuses a different payload, and commits the request and trade together.

5.3 Security boundary

Supabase Auth establishes identity. `profiles.role` distinguishes fans and organizers. RLS and grants restrict reads and writes. The mutation RPCs are the only write path. `resolve_market` additionally requires the organizer role. The public trade feed exposes recent trades without identity.

6. Domain model and invariants

The domain entities are matches, tournaments, registrations, profiles, markets, positions, trades, payouts and trade requests. The critical invariants are:

- INV-01: registrations never exceed `max_capacity`, including under concurrent requests.
- INV-02: a user cannot hold two registrations for the same tournament.
- M-INV1–M-INV8: market window, budget-bounded entry, ceil debit, floor credit, no negative balance, no oversell, one-shot resolution, no commission.
- M-INV9: a retried entry or exit with the same request key replays the recorded trade and does not duplicate a debit, credit, position or trade.
- INV-05 / A-INV1–A-INV5: authentication, RLS, least privilege and organizer-only resolution.

The authoritative status is the invariant registry. The key distinction is between **enforced** (the database has a mechanism) and **verified** (the mechanism was observed failing when deliberately weakened). Five market invariants and the M-INV7 detail set remain enforced but not fault-injected (FND-002).

7. Code map

Architecture element	Location
App Router pages	`src/app/page.tsx`, `src/app/login/page.tsx`, `src/app/market/[id]/page.tsx`, `src/app/tournaments/[id]/page.tsx`
Server Actions	`src/app/actions/auth.ts`, `market.ts`, `register.ts`
Client components	`src/components/MarketTrader.tsx`, `LoginForms.tsx`, `RegistrationForm.tsx`, `TopBar.tsx`
Session and Supabase clients	`src/lib/session.ts`, `src/lib/supabase/server.ts`, `src/lib/supabase/health.ts`
Health route	`src/app/api/health/route.ts`
Logging	`src/lib/logger.ts`

Architecture element	Location
Architecture guard	`scripts/check-architecture.ts`
Smoke test	`scripts/smoke-test.ts`
Migrations	`supabase/migrations/`
Invariant tests	`tests/invariants/`
CI	`.github/workflows/ci.yml`

The fuller mapping, including each migration and test, is `../04-code-map.md`.

8. Decisions and alternatives

Decision	Reason	Record
Next.js + Supabase + managed Postgres	One developer, two deadlines; database-enforced invariants; no infrastructure work	ADR-001
Live LMSR market with atomic entry/exit and one-shot resolution	The domain needs immediate, unambiguous point movement; eventual consistency would be wrong	ADR-002
Authentication, RLS, grants and `SECURITY DEFINER` RPCs	Direct table writes would bypass the business rules; the UI is not the authorization boundary	ADR-003
Request-key idempotency	A retry must not charge or pay twice; the cooldown is not a request key	Idempotency change

Rejected alternatives include a Spring Boot stack (not justified by the domain), self-hosted PostgreSQL (operational work without a measured driver), event-driven prediction processing (eventual consistency where the answer must be immediate), microservices (distributed cost without a driver) and event sourcing (extra operational surface without a requirement).

9. Evidence

Claim	Evidence	Limit
Capacity, balances, positions and resolution are correct under the recorded bursts	28 tests in CI; fault injection for INV-01, INV-02, M-INV4, M-INV6 and the M-INV7 gate	Local/CI, one database instance; five market invariants lack injection
Retries do not duplicate a trade	Six M-INV9 tests; request-key table	Legacy null-key path is not idempotent; no retention policy
Authorization boundary is enforced	Authorization suite: anonymous, fan and organizer boundaries	No production session/rate-limit hardening; club privacy rule not implemented
Burst p95 is below 1,000 ms	463.11 ms and 571.68 ms observed	Environment-specific; post-peak resource release unverified
Delivery is repeatable	`Delivery and recovery` job; migrations; smoke; runbook	Local/CI only; no production deployment

Claim	Evidence	Limit
Recovery works for the application schema	67,009-byte public-schema restore; tables 10→10; RLS policies 10→10; marker restored	Public schema only; no Supabase-managed auth/storage; no production RTO/RPO
Tests can fail	Fault injection records	Remaining invariants lack observed-red injection (FND-002)

CI run 42435627680 passed 28 tests with 0 failures. CI run 42467767299 passed the delivery and recovery drill.

10. Trade-offs and residual risk

The main trade-offs are Reliability vs Elasticity (pessimistic locking protects the invariant at the cost of peak throughput), Testability vs time to market (strong tests only for domain invariants), Deployability vs control (managed platform vs vendor coupling), and Security vs simplicity (a small RPC surface with a high-trust boundary).

Residual risks are recorded in the findings register and the residual risk register: FND-002 (remaining fault injections), FND-004 (p95 observed but not a hard gate), FND-006 (no hosted deployment or production recovery), FND-007 (no real organizer evidence), FND-008 (no independent review). FND-003 (idempotency) is fixed.

11. AI use

Generative AI assisted implementation, tests, documentation and review passes. The human project owner approved the plans, the product rules, the architecture decisions and the releases. The AI transparency log records each contribution, the verification performed and the limits. AI-generated code and tests can share assumptions; the independent business-rule oracle and the fault-injection evidence are the check, not the model's confidence.

12. Reproducibility

1. Start the local Supabase stack and replay migrations: `supabase start`, `supabase db reset`.
2. Write the local credentials to `.env.local`.
3. Run `bun install`, `bun run build`, `bun run test`, `bun run lint`, `bun run typecheck`, `bun run check:architecture`.
4. Start the app with `bun run start` and run `bun run smoke`.
5. Follow the operations runbook for release, rollback and recovery.

The detailed setup is in `../../../README.md`.

13. Limitations

- No hosted deployment, production TLS/DNS, external monitoring or alerting.

- No production backup/retention/RTO/RPO; the recovery drill covers the application-owned `public` schema only.
- No real organizer or fan observation; the problem claim is still a hypothesis.
- No independent human review.
- Five market invariants and the M-INV7 detail set remain without observed-red fault injection.
- The kata's user numbers are planning assumptions.
- The system is not a complete product: ticket purchase, rewards redemption and club analytics are described but not all implemented.

14. Related records

- Kata
- Architecture characteristics
- System overview
- Domain model
- C4 views
- Code map
- ADRs
- Invariant registry
- Findings register
- AI transparency log
- Operations runbook
- Methodology report

Appendix A — Task 1 and Task 2 submission map

Task	Source in this repository	What it contains	Submission note
Task 1 — Kata	`../kata.md`	Description, users/scale, K1–K15 requirements, additional context; one-page target	Export to PDF for the course submission.
Task 2 — Characteristics	`../03-architecture-characteristics.md`	Course-vocabulary mapping, selected characteristics, kata traceability, SMART targets, evidence and limits	Export to PDF; it may include the updated kata as the task slide permits.
Final architecture description	This report	Kata, characteristics, style, module boundaries, decisions, evidence, limits	The course permits PDF or Markdown; this report is Markdown with code references.

The two task sources are deliberately kept as separate documents so they can be exported without the full report. The report is the dated synthesis that connects them to the architecture description and the evidence.

Appendix B — What changed from the earlier task submission

Area	Earlier version	Current version
Kata requirements	Unnumbered bullets	K1–K15, referenced by the characteristics
Users	Types only	Types plus expected numbers/patterns and an explicit assumption label
Integrity characteristic	“Integrity under concurrency”	“Reliability — integrity under concurrency”, mapped to the course's operational vocabulary
Security	Selected as a first-class characteristic	Labelled as an implicit cross-cutting characteristic, with the course rationale
Elasticity target	Mixed capacity correctness and p95	Capacity correctness moved to Reliability; Elasticity measures burst response and post-peak resources
SMART	Implicit owner/time	Explicit owner, time horizon and review condition
Traceability	Domain concerns only	Characteristic → K points → why → target → evidence table
Deployability	Intended benefit	Measured local/CI job with migrations, smoke and recovery drill
Performance	Ambiguous overlap with Elasticity	Explicitly not selected separately; the only time target is the burst p95 under Elasticity

Appendix C — PDF export

No PDF toolchain is installed in the repository environment. The source documents are Markdown and can be exported with the course's preferred tool:

- a Markdown-to-PDF converter (for example Pandoc with a LaTeX or HTML engine);
- a browser print-to-PDF from a rendered Markdown preview;
- the university's document template, if one is required.

Task 1 and Task 2 require PDF submissions. The final architecture description may be PDF or Markdown. This report does not claim that a PDF has already been produced.

Update, 2026-11-08: the repository now has a PDF export pipeline (`bun run docs:pdf`, `scripts/export-pdf.ts`). It rendered Task 1, Task 2, this report and the methodology report into `docs/submissions/`. The paragraphs above describe the earlier state and are kept as the record.

15. MVP addendum — three roles and complete frontends

The first MVP adds the club role, organizer ownership, simulated tickets, local reward codes and complete fan/organizer/club frontends. The system remains one Next.js app and one

PostgreSQL database. Domain modules now include identity/roles, tournaments, marketplace/trading, tickets/rewards and club aggregate analytics.

- Authorization: `FAN`, `ORGANIZER` and `CLUB`; organizers own their tournaments/matches; clubs have aggregate-only RPC access and no private-table reads.
- Reliability: ticket uniqueness/allocation, reward balance, organizer ownership and market resolution have integration tests; the existing invariant/fault-injection suite remains green.
- Delivery: five CI jobs (build/security, invariants, recovery, fault injection, E2E/accessibility) plus a scheduled performance baseline.
- Limits: no hosted production deployment, no real payments/coupons, no independent privacy/security review, no real-user validation, and M-INV2 remains forced-failure rather than observed-red.